

Week 2 – Master Study Notes

Knowledge Representation, RDF/RDFS, SKOS & OWL Introduction

F20BD / F21BD – Big Data Management
Dr. Radu Mihailescu | Heriot-Watt University

Class Test Scope – READ FIRST

The **Week 7** class test covers:

- Introduction to Big Data
- **RDF / RDFS** ← **THIS LECTURE** – heavily tested!
- **SPARQL**
- **SKOS** ← **THIS LECTURE** – heavily tested!

NOT included in class test: OWL / Protégé (but **IS** in the final exam).

Throughout these notes, anything highlighted in **purple boxes** is **directly class-test relevant**.

Contents

1	The Big Picture – Why Do We Need All This?	2
1.1	Real-World Analogy: The Library	2
1.2	The Semantic Web Technology Stack (Simplified)	2
2	Knowledge Graphs – The Foundation of Everything	3
2.1	Real-World Analogy: A Family Tree	3
2.2	Five Key Characteristics of Knowledge Graphs	3
2.3	Reading a Graph: Triples	4
3	RDF – Resource Description Framework	5
3.1	What is an IRI?	5
3.2	What is a Literal?	5
3.3	RDF Serialisation Formats	5
3.4	Turtle Syntax – The Easy One	6
3.5	RDF/XML Syntax – The Verbose One	7
3.6	RDF Containers: Bag, Seq, Alt	9
3.7	RDF Collections – Closed Groups	10
3.8	Blank Nodes (Anonymous Nodes)	10
3.9	RDF Built-in Classes and Properties	12
3.10	Limitations of RDF – Why We Need RDFS	12
4	RDFS – RDF Schema (Vocabulary Layer)	13
4.1	RDFS Classes	13
4.2	RDFS Properties – The Power Tools	13
4.3	RDFS Metadata Properties	14
4.4	Schema vs Instance – Critical Distinction	14
4.5	Inference with RDFS	15
4.6	Standard Vocabularies Used with RDF/RDFS	16
4.7	RDFS Limitations – Why We Need More	16

5	SKOS – Simple Knowledge Organization System	17
5.1	SKOS Concepts and Labels	17
5.2	SKOS Relationships	17
5.3	Transitivity in SKOS – Tricky but Important	18
5.4	Disjoint Properties in SKOS	19
5.5	SKOS Matching – Linking Vocabularies	19
5.6	SKOS Reasoning Summary	20
5.7	SKOS Model Overview Diagram	20
6	OWL – Web Ontology Language (Final Exam Only)	21
6.1	Why OWL? – The Semantic Spectrum	21
6.2	Ontology – What is it?	21
6.3	TBox and ABox	21
6.4	The Meaning Triangle	22
6.5	OWL Key Features	22
6.5.1	Equivalent Classes	22
6.5.2	Disjoint Classes	22
6.5.3	Object Properties and Characteristics	22
6.6	Reasoning with OWL	23
6.7	Protégé – The Ontology Editor	23
6.8	Ontology Design Principles	23
7	Practice Questions	24
7.1	Question 1: Triples (Class Test Level)	24
7.2	Question 2: Inference (Class Test Level)	24
7.3	Question 3: SKOS Consistency (Class Test Level)	24
7.4	Question 4: Container vs Collection	24
7.5	Question 5: Domain and Range Inference	25
7.6	Question 6: Write Turtle from English	25
7.7	Question 7: OWL Reasoning (Final Exam Level)	25
8	Quick Reference Cheat Sheet	26

1 The Big Picture – Why Do We Need All This?

The Core Problem

The internet is full of documents written for *humans* to read. But computers can't truly *understand* what those documents mean. If you search Google for "apple", you might get results about the fruit, the tech company, or a music label. The computer doesn't know the difference – it just matches words.

The Semantic Web is a vision where data on the internet is structured so that machines can understand *meaning*, not just words.

1.1 Real-World Analogy: The Library

Imagine a library with millions of books, but:

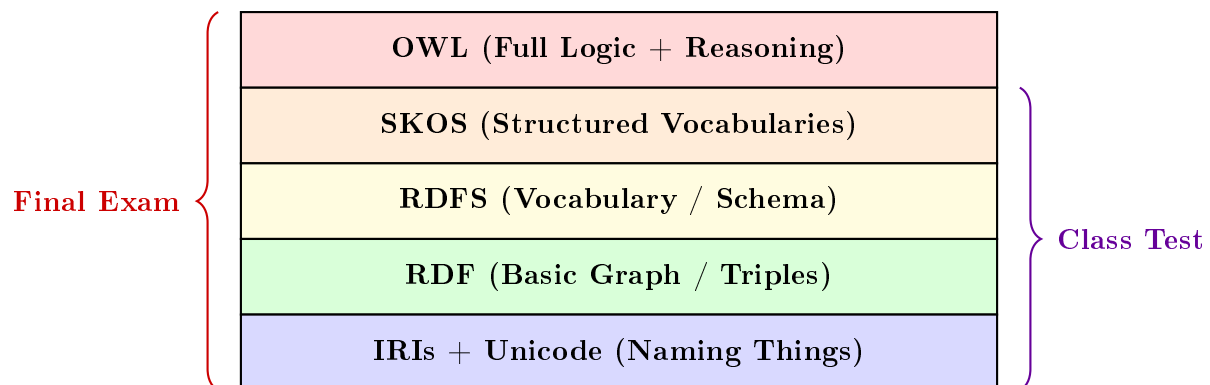
- Books have no titles on the spine
- There's no catalogue system
- Books are just piled randomly on shelves

That's basically the current web from a machine's perspective. **Semantic Web technologies** (RDF, RDFS, SKOS, OWL) are like creating a perfect catalogue system where every book is labelled, categorised, and linked to related books – and a robot librarian can navigate it all.



1.2 The Semantic Web Technology Stack (Simplified)

Think of it like building blocks, where each layer adds more power:



2 Knowledge Graphs – The Foundation of Everything

Knowledge Graphs are the basis of RDF

Understanding knowledge graphs is **essential** for the class test. Every RDF question is about reading or writing graph data.

What is a Knowledge Graph?

A **knowledge graph** is a way of representing knowledge as a **network (graph)** of connected things. It consists of:

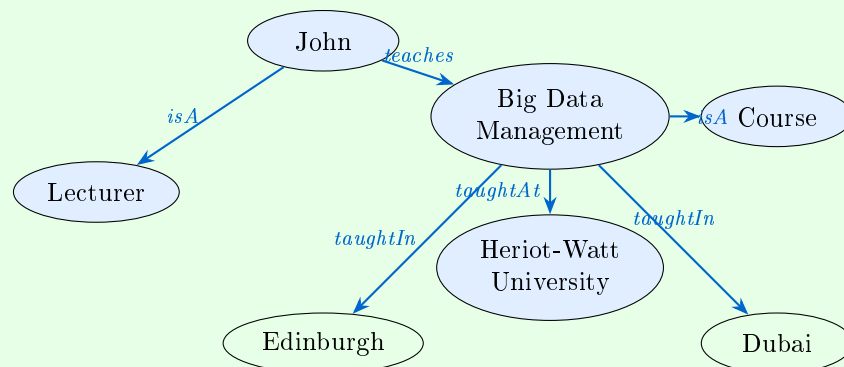
- **Nodes** = things (people, places, concepts, objects)
- **Edges (links)** = relationships between things

2.1 Real-World Analogy: A Family Tree

A family tree *is* a knowledge graph! Your mum is connected to you by the relationship “isMotherOf”. Your dad is connected to your mum by “isMarriedTo”. Every person is a **node**, and every relationship is an **edge**.

Example:

A Simple Knowledge Graph Let’s represent: “John is a lecturer who teaches Big Data Management at Heriot-Watt University, taught in Edinburgh and Dubai.”



2.2 Five Key Characteristics of Knowledge Graphs

Property	What It Means (ELI5)	Example
Scaling	One graph can hold knowledge about <i>many</i> different topics at once	Geography + People + Jobs all in one graph
Agreement	We need to agree on what relationships <i>mean</i>	Does “knows” mean “met once” or “is friends with”?
Structure	We need rules about which nodes connect to which	A “teaches” link should go from a Person to a Course
Plurality	The same relationship can appear many times	Multiple people can all have “isA → Person”
Asymmetry	Relationships have a <i>direction</i> (like one-way streets)	“John teaches BDM” ≠ “BDM teaches John”

2.3 Reading a Graph: Triples

Triple = The Atomic Unit of Knowledge

Every fact in a knowledge graph is stored as a **triple**:

(Subject, Predicate, Object)

- **Subject** = the thing you're talking about (source node)
- **Predicate** = the relationship (the edge/link)
- **Object** = the thing it connects to (destination node)

Think of it as a **sentence**: “John [subject] teaches [predicate] Big Data [object].”

Triple Practice The exam **will** ask you to read graphs and write triples, or to read triples and draw graphs. Practice both directions!

Example:

Converting a Sentence to Triples Sentence: “John is a lecturer who teaches Big Data at Heriot-Watt.”

Subject	Predicate	Object
John	isA	Lecturer
John	teaches	Big Data Management
Big Data Management	taughtAt	Heriot-Watt University

3 RDF – Resource Description Framework

RDF is CORE to the class test

You MUST know: what RDF is, how triples work, the different serialisation formats (especially Turtle and RDF/XML), blank nodes, containers (Bag, Seq, Alt), collections, and datatypes.

What is RDF?

RDF (Resource Description Framework) is a **standard language** for representing knowledge graphs. It's the foundation of the Semantic Web.

Key facts:

- Every piece of knowledge is a **triple**: (Subject, Predicate, Object)
- **Subject** and **Predicate** are always identified by an **IRI** (a web address like a URL)
- **Object** can be either an IRI (another resource) OR a **literal** (a plain value like a number or string)
- An RDF model = a collection of triples (unordered – the order doesn't matter)

3.1 What is an IRI?

IRI – Internationalized Resource Identifier

An IRI is like a **web address** that uniquely identifies something. It's basically a URL but for *any* resource (not just web pages).

Example: `http://example.org/people/John`

This IRI uniquely identifies “John” so that no matter who reads this data, they know *exactly* which “John” we mean. No confusion!

3.2 What is a Literal?

Literal = A Plain Value

A **literal** is just a value – a number, a date, a piece of text, etc. Literals have a **datatype** (from XML Schema – `xsd`).

Common datatypes:

Datatype	What It Is	Example
<code>xsd:string</code>	Text	“Hello World”
<code>xsd:integer</code>	Whole number	42
<code>xsd:float</code>	Decimal number	3.14
<code>xsd:boolean</code>	True or false	true
<code>xsd:date</code>	A date	“2024-01-15”
<code>xsd:dateTime</code>	Date + time	“2024-01-15T10:30:00”

3.3 RDF Serialisation Formats

RDF is a *data model* (an idea). To actually *write it down*, we use a **serialisation format**. Think of it like this: the number “five” is a concept, but you can write it as “5”, “V”, “five”, or “cinq” – different formats, same meaning.

Know These Two Formats

For the class test, you MUST be able to read and write both **RDF/XML** and **Turtle** syntax. These are the two most commonly tested formats.

Format	Description	Exam?
RDF/XML	Original XML-based format. Verbose but widely used.	YES
Turtle	Compact, human-friendly format. Easy to read/write.	YES
N-Triples	One triple per line. Very simple.	Maybe
JSON-LD	JSON-based. Good for web developers.	Unlikely

3.4 Turtle Syntax – The Easy One

Turtle Format Rules

Turtle is the **most human-readable** RDF format. Here are the rules:

1. **Prefixes** are declared at the top with **@prefix**
2. Each triple is: **subject predicate object .** (note the **dot** at the end!)
3. Use **;** to add another predicate to the *same* subject
4. Use **,** to add another object to the *same* predicate
5. **a** is a shortcut for **rdf:type** (means “is an instance of”)
6. IRIs go inside **<angle brackets>** or use **prefix:name**
7. Literals go inside **"double quotes"**

Example:

Turtle Step-by-Step Let’s say we want to represent:

- John is a Person
- John’s name is “John Smith”
- John’s age is 30
- John knows Mary
- John knows Bob

```
@prefix ex: <http://example.org/> .
@prefix foaf: <http://xmlns.com/foaf/0.1/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ex:John a foaf:Person ;          # John is a Person
    foaf:name "John Smith" ;      # John's name is "John Smith"
    foaf:age "30"^^xsd:integer ;  # John's age is 30
    foaf:knows ex:Mary ,          # John knows Mary AND
                                ex:Bob .      # John knows Bob (note the DOT = end)
```

Breaking it down:

- Line 1–3: We define shortcuts (prefixes) so we don’t type full URLs every time
- **a** = means “is a type of” (same as **rdf:type**)
- **;** = “same subject, new predicate” (still talking about John)
- **,** = “same subject AND same predicate, new object” (John knows Mary AND Bob)
- **.** = “I’m done talking about this subject”
- **“30”^^xsd:integer** = the number 30 (with its datatype)

Turtle Punctuation Cheat Sheet

Symbol	Meaning	Memory Trick
.	End of this subject's triples	Full stop = "I'm done"
;	Same subject, different predicate	Semi = "more to say about same thing"
,	Same subject & predicate, different object	Comma = "and also this one"
a	Shortcut for <code>rdf:type</code>	"is a" type of
[]	Blank node (anonymous)	"some unnamed thing"

3.5 RDF/XML Syntax – The Verbose One

RDF/XML Format

RDF/XML is the **original** format. It uses XML tags. It's more verbose (longer to write) but you **MUST** be able to read it.

Key elements:

Element	What It Does
<code><rdf:RDF></code>	Root element – wraps everything
<code><rdf:Description rdf:about="..."></code>	Describes a resource (the subject)
<code>rdf:about</code>	Identifies which existing resource we're talking about
<code>rdf:ID</code>	Creates a <i>new</i> identifier (relative to base)
<code>rdf:resource</code>	Points to another resource (makes the object an IRI, not a literal)
<code>rdf:type</code>	Says what class/type something belongs to

Example:

Same Data in RDF/XML The same John example in RDF/XML:

```
<?xml version="1.0"?>
<rdf:RDF
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:foaf="http://xmlns.com/foaf/0.1/"
  xmlns:ex="http://example.org/">

  <rdf:Description rdf:about="http://example.org/John">
    <rdf:type rdf:resource="http://xmlns.com/foaf/0.1/Person"/>
    <foaf:name>John Smith</foaf:name>
    <foaf:age rdf:datatype="http://www.w3.org/2001/XMLSchema#integer">
      30
    </foaf:age>
    <foaf:knows rdf:resource="http://example.org/Mary"/>
    <foaf:knows rdf:resource="http://example.org/Bob"/>
  </rdf:Description>

</rdf:RDF>
```

Key observations:

- Namespaces (prefixes) are declared in the `<rdf:RDF>` tag using `xmlns`:
- `rdf:about="..."` tells us WHO we are describing (the subject)
- When the object is **another resource**, we use `rdf:resource="..."`

- When the object is a **literal** (plain text/number), we just put it between the tags
- `rdf:datatype` specifies the type of the literal

△ **Warning:**

`rdf:about` vs `rdf:ID`

- `rdf:about` = “I’m talking about this *existing* thing” (full IRI or reference)
- `rdf:ID` = “I’m *creating* a new identifier” (gets appended to the base URI with a #)

Example: If base is `http://example.org/`, then `rdf:ID="snack"` creates `http://example.org/#snack`

3.6 RDF Containers: Bag, Seq, Alt

Containers are frequently tested

You will likely need to distinguish between Bag, Seq, and Alt, and write/read them in both Turtle and RDF/XML.

Sometimes you need to group multiple things together. RDF gives you three types of **containers**:

Container	Meaning	Ordered?	Analogy
rdf:Bag	An unordered group	No	A bag of marbles (any order)
rdf:Seq	An ordered group	Yes	A playlist (order matters)
rdf:Alt	A set of alternatives (pick ONE)	No	A menu (choose one dish)

△ Warning:

Containers are OPEN Containers can have *additional* members added later. They are not “sealed.” Anyone can add more items. This is an important distinction from **Collections** (see below).

Example:

rdf:Bag – Beatles Members (Unordered) The Beatles have 4 members, and we don’t care about the order:

Turtle:

```
@prefix cd: <http://www.recshop.fake/cd#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .

<http://www.recshop.fake/cd/Beatles> cd:artist [
  a rdf:Bag ;
  rdf:_1 "John" ;
  rdf:_2 "Paul" ;
  rdf:_3 "George" ;
  rdf:_4 "Ringo"
] .
```

RDF/XML:

```
<rdf:Description rdf:about="http://www.recshop.fake/cd/Beatles">
  <cd:artist>
    <rdf:Bag>
      <rdf:li>John</rdf:li>
      <rdf:li>Paul</rdf:li>
      <rdf:li>George</rdf:li>
      <rdf:li>Ringo</rdf:li>
    </rdf:Bag>
  </cd:artist>
</rdf:Description>
```

Note: **rdf:li** in XML auto-numbers to **rdf:_1**, **rdf:_2**, etc. in the actual RDF model.

Example:

rdf:Seq – Beatles Members (Ordered) Same members, but now order matters (e.g., order they joined):

The code is **identical** except you replace **rdf:Bag** with **rdf:Seq**. The numbering (**rdf:_1**, **rdf:_2**, ...) now carries meaning – first, second, third, etc.

Example:

rdf:Alt – Available Formats (Pick One) The Beatles album is available on CD, Record, or Tape – you pick ONE format:

Turtle:

```
<http://www.recshop.fake/cd/Beatles> cd:format [
  a rdf:Alt ;
  rdf:_1 "CD" ;
  rdf:_2 "Record" ;
  rdf:_3 "Tape"
] .
```

3.7 RDF Collections – Closed Groups

Collection vs Container

A **Collection** (rdf:parseType="Collection" in XML) is a **closed** group – it can **ONLY** contain the specified members. No one can add more later.

Memory trick:

- Container = open box (anyone can throw stuff in)
- Collection = sealed envelope (fixed members, that's it)

Example:

Collection in RDF/XML

```
<rdf:Description rdf:about="http://www.recshop.fake/cd/Beatles">
  <cd:artist rdf:parseType="Collection">
    <rdf:Description rdf:about="cd:George"/>
    <rdf:Description rdf:about="cd:John"/>
    <rdf:Description rdf:about="cd:Paul"/>
    <rdf:Description rdf:about="cd:Ringo"/>
  </cd:artist>
</rdf:Description>
```

Key difference: uses rdf:parseType="Collection" and each member is a rdf:Description (a resource with its own IRI), not just a string.

Container vs Collection – Exam Favourite!

	Container (Bag/Seq/Alt)	Collection
Open/Closed?	Open (can add members)	Closed (fixed members)
XML syntax	<rdf:Bag>, <rdf:Seq>, <rdf:Alt>	rdf:parseType="Collection"
Members	<rdf:li>	<rdf:Description>

3.8 Blank Nodes (Anonymous Nodes)

Blank Nodes

You may be asked to identify or create blank nodes. Know what they are and when to use them.

Blank Node

A **blank node** is a node in the graph that **doesn't have an IRI**. It's anonymous – it exists, but we don't give it a public name.

When to use: When a resource doesn't need to be referenced from outside. It's just a "helper" node.

Analogy: Imagine writing "John knows someone whose surname is Smith." You don't know that someone's name/ID – they're a blank node.

Example:

Blank Node – John knows an unnamed person **Turtle:**

```
@prefix pep: <http://example.com/people#> .

pep:John pep:knows [
  pep:surname "Smith" ;
  pep:givenName "Paul" ;
  pep:email "paul.smith@example.com"
] .
```

The [...] is a blank node. It has properties (surname, givenName, email) but no IRI. The system generates an internal ID like `_:b1` to track it.

RDF/XML:

```
<rdf:Description rdf:about="pep:John">
  <pep:knows>
    <rdf:Description>          <!-- No rdf:about = blank node! -->
      <pep:surname>Smith</pep:surname>
      <pep:givenName>Paul</pep:givenName>
      <pep:email>paul.smith@example.com</pep:email>
    </rdf:Description>
  </pep:knows>
</rdf:Description>
```

Notice: the inner `<rdf:Description>` has **no** `rdf:about` attribute – that's what makes it blank!

What Can Be in a Triple?

	IRI	Literal	Blank Node
Subject	✓	✗	✓
Predicate	✓	✗	✗
Object	✓	✓	✓

Key rule: Predicates can **ONLY** be IRIs. Subjects can be IRIs or blank nodes. Objects can be anything.

3.9 RDF Built-in Classes and Properties

RDF Classes	What It Represents
<code>rdf:Property</code>	The class of all properties/relationships
<code>rdf:Bag</code>	An unordered container
<code>rdf:Seq</code>	An ordered container
<code>rdf:Alt</code>	A container of alternatives
<code>rdf:List</code>	A linked list structure
<code>rdf:Statement</code>	A triple (used for reification – talking about triples)
<code>rdf:XMLLiteral</code>	XML content as a literal

RDF Properties	What It Does
<code>rdf:type</code>	Says what class an instance belongs to (“is a”)
<code>rdf:first</code>	First element of a list
<code>rdf:rest</code>	Remaining elements of a list
<code>rdf:value</code>	A principal value of a resource
<code>rdf:subject</code>	Subject of a reified statement
<code>rdf:predicate</code>	Predicate of a reified statement
<code>rdf:object</code>	Object of a reified statement

3.10 Limitations of RDF – Why We Need RDFS

△ Warning:

RDF Can't Enforce Rules Plain RDF has **no type system**. It cannot:

- Stop you from using “lastName” as a property of a car (makes no sense!)
- Define which relationships are allowed between which types of things
- Enforce that a “teaches” link should only go from a Person to a Course

This leads to two problems:

1. **Interpretability** – Different people understand predicates differently
2. **Automation** – Software can't check if a knowledge graph makes sense

Solution: RDFS adds a “schema” layer on top of RDF to define rules and vocabulary.

4 RDFS – RDF Schema (Vocabulary Layer)

RDFS is CORE to the class test

You must understand: RDFS classes, RDFS properties (subClassOf, subPropertyOf, domain, range), the Schema vs Instance distinction, and basic inference.

What is RDFS?

RDFS (RDF Schema) is an **extension of RDF** that lets you define the **vocabulary** (the “dictionary”) for your knowledge graph.

Think of it this way:

- **RDF** = the sentences (the data/facts)
- **RDFS** = the grammar rules (what words mean and how they can be used)

RDFS lets you:

1. Define **classes** (types of things: Person, Animal, Course...)
2. Create **hierarchies** (a Dog is a type of Animal)
3. Set **domain and range** (“teaches” goes from Person to Course)
4. Enable basic **inference** (if Dog is an Animal, and Rex is a Dog, then Rex is an Animal)

4.1 RDFS Classes

RDFS Class	What It Is (ELI5)	Analogy
<code>rdfs:Class</code>	The “class of all classes” – defines what a class is	The concept of “category” itself
<code>rdfs:Resource</code>	Everything is a resource. The ultimate parent class.	“Thing” (everything is a thing)
<code>rdfs:Literal</code>	The class of plain values (strings, numbers, dates)	Numbers and text
<code>rdfs:Datatype</code>	The class of datatypes (integer, string, etc.)	“Types of values”

△ Warning:

Don’t Confuse These!

- `rdf:resource` (lowercase r) = an XML **attribute** used to point to another IRI in RDF/XML
- `rdfs:Resource` (capital R) = a **class** that represents all resources

These are completely different things! The exam may try to trick you.

4.2 RDFS Properties – The Power Tools

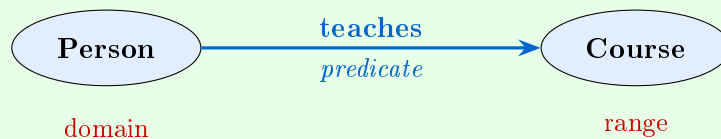
These four properties are the heart of RDFS

`rdfs:subClassOf`, `rdfs:subPropertyOf`, `rdfs:domain`, and `rdfs:range` are the most tested RDFS concepts.

Property	What It Does (ELI5)	Example
<code>rdfs:subClassOf</code>	“A is a <i>kind of</i> B” – creates a class hierarchy. Transitive!	Horse subClassOf Animal
<code>rdfs:subPropertyOf</code>	“Relationship A is a <i>special case of</i> relationship B.” Transitive!	hasDaughter subPropertyOf hasChild
<code>rdfs:domain</code>	“This property’s <i>subject</i> must be of this class”	teaches → domain = Person
<code>rdfs:range</code>	“This property’s <i>object</i> must be of this class (or datatype)”	teaches → range = Course

Example:

Domain and Range Visualised



This means: if someone uses the **teaches** property, the **subject must be a Person** and the **object must be a Course**. If you write `ex:MyCar ex:teaches ex:Math`, a reasoner will *infer* that MyCar is a Person (which is probably wrong – that’s how domain/range catch mistakes).

4.3 RDFS Metadata Properties

These are used to add human-readable information:

Property	Purpose
<code>rdfs:label</code>	A human-friendly name for the resource
<code>rdfs:comment</code>	A longer text description
<code>rdfs:isDefinedBy</code>	Links to where this resource is formally defined
<code>rdfs:seeAlso</code>	Links to a related resource for more info

4.4 Schema vs Instance – Critical Distinction**Schema vs Instance**

	Schema (TBox)	Instance (ABox)
What?	The types of things and rules	The actual things
Contains	Classes, properties, domain/range, hierarchies	Individuals, real data
Analogy	The blueprint for a house	The actual house built from the blueprint
Example	“Person is a class. teaches goes from Person to Course.”	“John is a Person. John teaches BDM.”

Example:Full RDFS Example – Animals **Schema (RDFS)**: Define the vocabulary

```
@base <http://www.animals.fake/animals> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix aml: <http://www.animals.fake/animals#> .

aml:animal    a rdfs:Class .           # "animal" is a class
aml:horse     a rdfs:Class ;           # "horse" is a class
              rdfs:subClassOf aml:animal . # horse is a KIND OF animal
aml:camel     a rdfs:Class ;           # "camel" is a class
              rdfs:subClassOf aml:animal . # camel is a KIND OF animal
```

Instance (RDF data): Use the vocabulary

```
@prefix aml: <http://www.animals.fake/animals#> .
@prefix ex: <http://example.org/> .

ex:Spirit     a aml:horse ;           # Spirit is a horse
              aml:name "Spirit" .
ex:Humphrey   a aml:camel ;           # Humphrey is a camel
              aml:name "Humphrey" .
```

What a reasoner infers: Since `horse subClassOf animal`, and `Spirit` is a horse, the reasoner concludes:

`ex:Spirit a aml:animal .` (Spirit is also an animal!) – this was never explicitly stated!

4.5 Inference with RDFS

Inference Questions

The class test may ask you to identify what new triples can be **inferred** (discovered) from existing ones.

Inference = Discovering Hidden Facts

Inference means discovering facts that are **NOT explicitly stated** by applying logical rules. RDFS supports two main types:

1. **Type Inheritance:** Following the class hierarchy upward
2. **Type Inference:** Using domain/range constraints to figure out types

Example:

Type Inheritance Given:

```
:alice rdf:type foaf:Person .
foaf:Person rdfs:subClassOf foaf:Agent .
```

Inferred: `:alice rdf:type foaf:Agent .`**Why?** Alice is a Person. Person is a subclass of Agent. Therefore Alice is also an Agent.**Analogy:** If Alice is a Student, and all Students are People, then Alice is a Person.**Example:**

Type Inference via Domain/Range Given:

```
foaf:knows rdfs:domain foaf:Person .
foaf:knows rdfs:range foaf:Person .
```

```
:alice foaf:knows :bob .
```

Inferred:

```
:alice rdf:type foaf:Person .      # alice is in the domain
:bob  rdf:type foaf:Person .      # bob is in the range
```

Why? The *knows* property has domain = Person and range = Person. Since Alice *knows* Bob, Alice must be a Person (domain) and Bob must be a Person (range).

4.6 Standard Vocabularies Used with RDF/RDFS

Know These Vocabularies

You should know what each vocabulary is for and recognise their prefixes.

Vocabulary	Prefix	Purpose
RDF	rdf:	Core graph model (triples, types, containers)
RDFS	rdfs:	Schema layer (classes, properties, hierarchies)
Dublin Core	dc: or dcterms:	Basic metadata: title, creator, description, date
SKOS	skos:	Structured vocabularies, taxonomies, thesauri
FOAF	foaf:	People, their activities, and social connections
Schema.org	schema:	Web-wide vocabulary for structured data

Example:

Dublin Core – Simple Metadata Dublin Core defines basic metadata properties:

```
@prefix dc: <http://purl.org/dc/elements/1.1/> .

ex:myReport dc:title "Annual Report 2024" ;
            dc:creator "John Smith" ;
            dc:description "Yearly summary of operations" ;
            dc:date "2024-12-01" .
```

4.7 RDFS Limitations – Why We Need More

RDFS is good but limited. It **cannot**:

- Say that range of *hasChild* is Person when applied to Persons, but Elephant when applied to Elephants (localised constraints)
 - Say that every Person has exactly 2 parents (cardinality)
 - Say that *isPartOf* is transitive, or *hasPart* is inverse of *isPartOf*
 - Provide powerful reasoning
- ⇒ This is why we need **OWL** (covered later, in the final exam).

5 SKOS – Simple Knowledge Organization System

SKOS is IN the class test

SKOS is explicitly listed in the class test topics. Know: concepts, labels, relationships (broader/-narrower/related), transitivity, disjoint properties, and matching.

What is SKOS?

SKOS is a standard RDF vocabulary (since 2009) designed to represent **structured vocabularies** like:

- **Taxonomies** (classification systems, like how species are organised)
- **Thesauri** (lists of terms with synonyms and relationships)
- **Subject headings** (library classification)

Real-world analogy: Think of how a library organises books. “Science” is a broad category. Under it, you have “Biology”, “Chemistry”, “Physics”. Under Biology, you have “Zoology”, “Botany”. SKOS is the system for creating this kind of organised hierarchy.

5.1 SKOS Concepts and Labels

SKOS Concept

The building block of SKOS is the **Concept** – a “unit of thought”. It’s like a word in a dictionary. Each concept has **labels**:

Label Type	What It Is	How Many?
skos:prefLabel	The preferred name	Exactly 1 per language
skos:altLabel	Alternative names (synonyms)	Any number
skos:hiddenLabel	Hidden search terms	Any number

Example:

Creating a SKOS Concept

```
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix ex: <http://www.example.com/> .

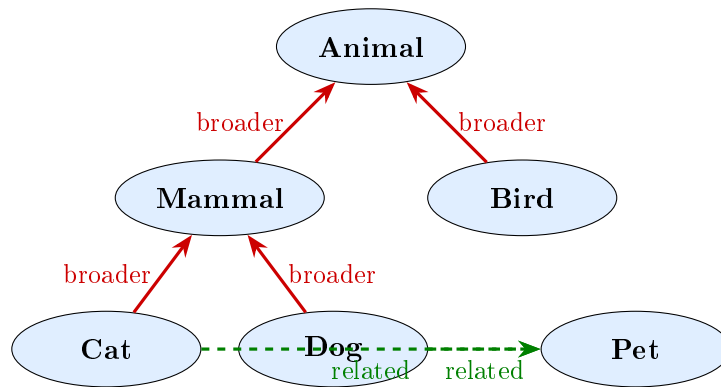
ex:cat a skos:Concept ;
      skos:prefLabel "cat"@en ;           # preferred English name
      skos:prefLabel "chat"@fr ;         # preferred French name
      skos:altLabel "feline"@en ;        # synonym in English
      skos:altLabel "kitty"@en .         # another synonym
```

5.2 SKOS Relationships

SKOS Relationships – High Priority

You **MUST** know the three relationship types, the difference between broader/narrower and broaderTransitive/narrowerTransitive, and the disjointness rule.

SKOS defines three types of relationships between concepts:



Relationship	Meaning	Type	Analogy
skos:broader	A is a more general concept than me	Hierarchical	Cat → broader → Animal
skos:narrower	A is a more specific concept than me	Hierarchical	Animal → narrower → Cat
skos:related	A is associated with me (not a hierarchy)	Associative	Cat ↔ related ↔ Pet

Tip:

broader and narrower are inverses If you say `:cat skos:broader :mammal`, a reasoner can infer `:mammal skos:narrower :cat`.

5.3 Transitivity in SKOS – Tricky but Important

Transitive vs Not Transitive

A relationship is **transitive** if: when $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$ automatically.

Analogy: “Taller than” is transitive. If Alice is taller than Bob, and Bob is taller than Charlie, then Alice is taller than Charlie.

CRITICAL FACT: `skos:broader` and `skos:narrower` are **NEITHER** transitive **NOR** intransitive. They don’t tell you anything about indirect relationships.

For explicit transitivity, use:

- `skos:broaderTransitive` – guaranteed transitive version of broader
- `skos:narrowerTransitive` – guaranteed transitive version of narrower

Example:

Transitivity Example

```

:animal a skos:Concept .
:mammal a skos:Concept ; skos:broaderTransitive :animal .
:cat a skos:Concept ; skos:broaderTransitive :mammal .
  
```

Inferred: `:cat skos:broader :animal` .

Because `broaderTransitive` IS transitive: $\text{cat} \rightarrow \text{mammal} \rightarrow \text{animal}$, so $\text{cat} \rightarrow \text{animal}$.

Three Flavours of Transitivity

Type	Rule	Example
Transitive	$A \rightarrow B$ and $B \rightarrow C \Rightarrow A \rightarrow C$	broaderTransitive
Intransitive	$A \rightarrow B$ and $B \rightarrow C \Rightarrow \text{NOT } A \rightarrow C$	(none in SKOS)
Neither	$A \rightarrow B$ and $B \rightarrow C \Rightarrow \text{unknown}$	broader, narrower, related

5.4 Disjoint Properties in SKOS

Disjoint = Cannot Happen Together

In SKOS, `skos:related` is **disjoint** with both `skos:broader` and `skos:narrower`. This means: **two concepts cannot be both “related” and “broader/narrower” at the same time.**

Analogy: You can’t be someone’s boss (hierarchical) and also their “colleague” (peer/associative) in the exact same relationship. Pick one.

Example:

Disjoint Inconsistency – Exam Favourite!

```
:r1 a skos:Concept ; skos:related :r3 .
:r2 a skos:Concept ; skos:narrowerTransitive :r1 .
:r3 a skos:Concept ; skos:narrowerTransitive :r2 .
```

Problem: By transitivity of `narrowerTransitive`:

`:r3 skos:narrower :r1` (since $r3 \rightarrow r2 \rightarrow r1$).

But we also said `:r1 skos:related :r3`.

Now `r1` and `r3` are both **related** AND **narrower** – but these are **disjoint**!

$\Rightarrow$ This is **INCONSISTENT**. A reasoner would flag this as an error.

5.5 SKOS Matching – Linking Vocabularies

SKOS Matching Properties

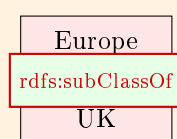
These are used to say “this concept in vocabulary A is similar to that concept in vocabulary B”:

Property	Meaning	Strength
<code>skos:exactMatch</code>	Same concept, fully interchangeable	Strong
<code>skos:closeMatch</code>	Almost the same, interchangeable in <i>some</i> cases	Medium
<code>owl:sameAs</code>	Literally the <i>same thing</i> (same identity)	Strongest

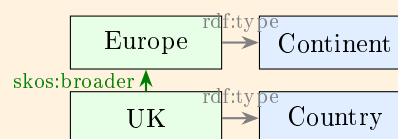
△ Warning:

Common Mistake with SKOS Hierarchies **Do NOT** use `rdfs:subClassOf` for SKOS concepts!

WRONG:



CORRECT:



UK is NOT a “subclass” of Europe. UK is a **concept that is narrower than** Europe. Use

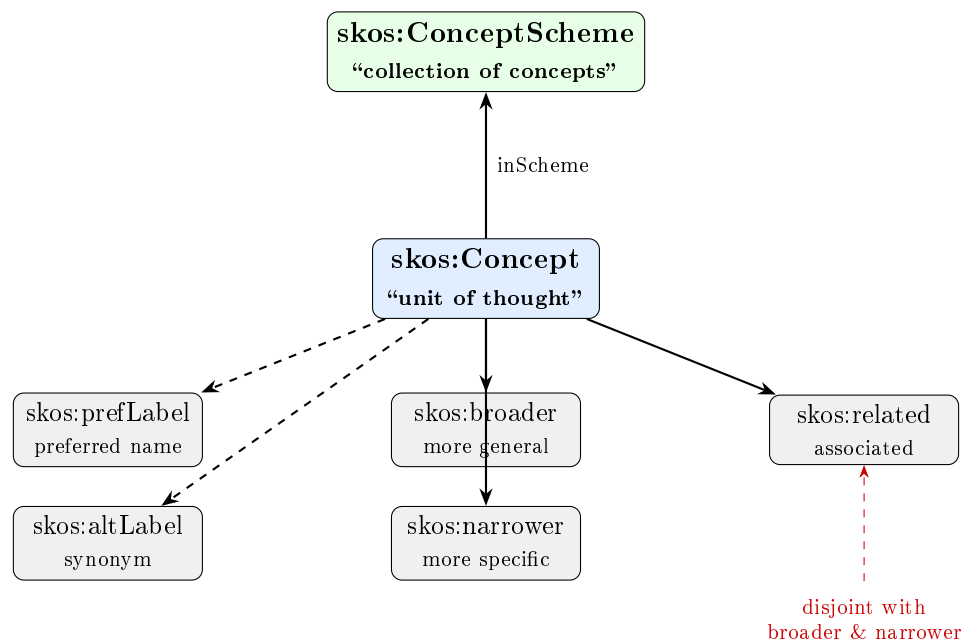
`skos:broader/skos:narrower` for concept hierarchies.

5.6 SKOS Reasoning Summary

With SKOS, a reasoner can:

1. **Check consistency:** e.g., X broader than Y and Y broader than X at the same time = inconsistent
2. **Trivial entailments:** X broader than Y $\Rightarrow$ Y narrower than X
3. **Transitivity:** X narrowerTransitive Y, Y narrowerTransitive Z $\Rightarrow$ X narrower Z
4. **Disjointness checking:** related + broader/narrower between same concepts = error

5.7 SKOS Model Overview Diagram



6 OWL – Web Ontology Language (Final Exam Only)

△ Warning:

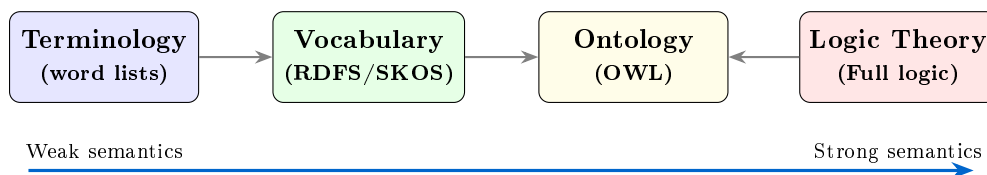
NOT in the Class Test OWL is **NOT** included in the Week 7 class test. However, it **IS** in the final exam. Study this section for the final exam.

What is OWL?

OWL (Web Ontology Language) is the most powerful semantic web language. It extends RDFS to allow:

- **Equivalence:** “Human and Person are the same class”
- **Disjointness:** “Man and Woman cannot overlap”
- **Property characteristics:** Symmetric, transitive, inverse, functional
- **Cardinality:** “A person has exactly 2 parents”
- **Full reasoning:** Consistency checking, classification, realisation

6.1 Why OWL? – The Semantic Spectrum



6.2 Ontology – What is it?

Ontology (ELI5)

An **ontology** is a formal description of knowledge about a domain. It defines:

- **Concepts (Classes):** types of things (Person, Animal, Vehicle)
- **Properties:** what you can say about those things
 - **Object Properties:** relationships between things (`marriedTo`, `teaches`)
 - **Data Properties:** attributes with values (`hasAge`, `hasName`)
- **Constraints (Axioms):** rules about properties (cardinality, domain/range)
- **Individuals:** actual things (John, Mary)

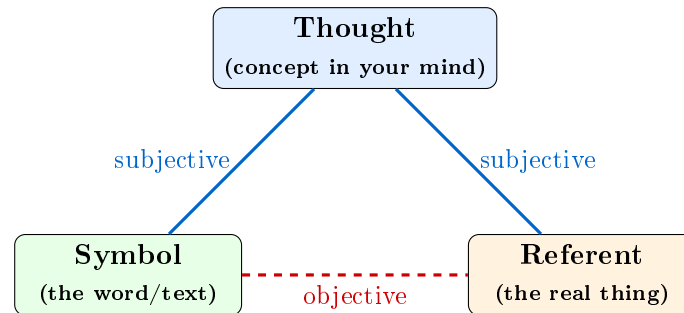
When an ontology includes actual data (individuals), it becomes a **Knowledge Base**.

6.3 TBox and ABox

TBox and ABox

	What	Analogy
TBox	The ontology – classes, relationships, rules	The recipe
ABox	The data – actual individuals/instances	The cake made from the recipe

6.4 The Meaning Triangle



The word “camel” (symbol) connects to your mental image (thought) and the actual animal (referent). The link between symbol and referent is **objective** – we need to agree on it. Ontologies make this agreement **formal and machine-readable**.

6.5 OWL Key Features

6.5.1 Equivalent Classes

```
:Human owl:equivalentClass :Person .
```

Means: Human and Person refer to *exactly the same things*. Every Human is a Person and every Person is a Human.

6.5.2 Disjoint Classes

```
:Man owl:disjointWith :Woman .
```

Means: No individual can be both a Man and a Woman at the same time.

For multiple disjoint classes:

```
[ rdf:type owl:AllDisjointClasses ;
  owl:members ( :Cat :Dog :Bird ) ] .
```

6.5.3 Object Properties and Characteristics

Characteristic	Meaning	Example
Symmetric	If $A \rightarrow B$ then $B \rightarrow A$	“marriedTo”
Asymmetric	If $A \rightarrow B$ then NOT $B \rightarrow A$	“isParentOf”
Transitive	$A \rightarrow B, B \rightarrow C \Rightarrow A \rightarrow C$	“isAncestorOf”
Inverse	Property A is the reverse of property B	“hasWife” inverse of “hasHusband”
Functional	Each subject has at most ONE value	“hasBirthDate” (you have only one)

Example:

OWL in Turtle – Putting It Together

```
:hasWife rdf:type owl:ObjectProperty ;
         rdfs:subPropertyOf :hasSpouse ;
         rdf:type owl:AsymmetricProperty ;
         owl:inverseOf :hasHusband ;
         rdfs:domain :Man ;
         rdfs:range :Woman .
```

```
:John rdf:type owl:NamedIndividual, :Man ;
      :hasWife :Mary .
```

What a reasoner infers:

- `:Mary :hasHusband :John` (because `hasWife` is inverse of `hasHusband`)
- `:Mary rdf:type :Woman` (because range of `hasWife` is `Woman`)
- `:John :hasSpouse :Mary` (because `hasWife` subPropertyOf `hasSpouse`)

6.6 Reasoning with OWL

A reasoner (like HermiT or Pellet in Protégé) can:

Task	What It Does
Consistency checking	Checks for contradictions (e.g., John is both Man and Woman, but they're disjoint)
Concept satisfiability	Checks if a class <i>could</i> have any members
Classification	Builds the full class hierarchy (discovers missing subClassOf links)
Realisation	Figures out what class each individual belongs to

6.7 Protégé – The Ontology Editor

Protégé is a free, open-source tool for building OWL ontologies. Steps to create an ontology:

1. Set the IRI (unique identifier for your ontology)
2. Add metadata (title, description, version)
3. Define prefixes
4. Create the class hierarchy
5. Add object properties (relationships between classes)
6. Add data properties (attributes like name, age)
7. Define constraints (domain, range, cardinality, characteristics)
8. Add individuals (actual data instances)
9. Run a reasoner to check consistency and discover inferences

6.8 Ontology Design Principles

Tip:

Best Practices for Ontology Creation

- There is **no single correct** model – the model should serve your application
- Ontology design is **iterative** – you refine multiple times
- Use **nouns** for class names, **verbs** for property names
- Consider **reusing** existing ontologies before creating new ones
- Follow the process: scope → reuse → terms → classes → properties → constraints → instances

7 Practice Questions

7.1 Question 1: Triples (Class Test Level)

Given the following Turtle code, list all the triples:

```
@prefix ex: <http://example.org/> .
@prefix foaf: <http://xmlns.com/foaf/0.1/> .

ex:Alice a foaf:Person ;
         foaf:name "Alice Smith" ;
         foaf:knows ex:Bob .
ex:Bob   a foaf:Person .
```

Answer:

Subject	Predicate	Object
ex:Alice	rdf:type	foaf:Person
ex:Alice	foaf:name	"Alice Smith"
ex:Alice	foaf:knows	ex:Bob
ex:Bob	rdf:type	foaf:Person

7.2 Question 2: Inference (Class Test Level)

Given:

```
:Dog rdfs:subClassOf :Animal .
:Animal rdfs:subClassOf :LivingThing .
:Rex rdf:type :Dog .
```

What can a reasoner infer?

Answer:

1. :Rex rdf:type :Animal (because Dog subClassOf Animal)
2. :Rex rdf:type :LivingThing (because Animal subClassOf LivingThing – transitivity!)
3. :Dog rdfs:subClassOf :LivingThing (subClassOf is transitive)

7.3 Question 3: SKOS Consistency (Class Test Level)

Is the following consistent? Explain.

```
:Biology skos:broader :Science .
:Science skos:related :Biology .
```

Answer: **INCONSISTENT**. skos:related is disjoint with skos:broader (and skos:narrower). Since Biology is broader than Science, the reasoner infers :Science skos:narrower :Biology. Now we have both related and narrower between Science and Biology, which violates the disjointness constraint.

7.4 Question 4: Container vs Collection

What is the key difference between an RDF Container and an RDF Collection?

Answer:

- **Container** (Bag, Seq, Alt): **Open** – additional members can be added by anyone at any time
- **Collection**: **Closed** – contains ONLY the specified members, no additions allowed

7.5 Question 5: Domain and Range Inference

Given:

```
:teaches rdfs:domain :Lecturer .
:teaches rdfs:range :Course .
:John :teaches :BDM .
```

What is inferred?

Answer:

- `:John rdf:type :Lecturer` (John is the subject of teaches, domain = Lecturer)
- `:BDM rdf:type :Course` (BDM is the object of teaches, range = Course)

7.6 Question 6: Write Turtle from English

Represent the following in Turtle: “Edinburgh is a city in Scotland. Scotland is a country. Edinburgh has a population of 500,000.”

Answer:

```
@prefix ex: <http://example.org/> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ex:Edinburgh a ex:City ;
    ex:locatedIn ex:Scotland ;
    ex:population "500000"^^xsd:integer .

ex:Scotland a ex:Country .
```

7.7 Question 7: OWL Reasoning (Final Exam Level)

Given:

```
:hasWife owl:inverseOf :hasHusband .
:hasWife rdfs:domain :Man ; rdfs:range :Woman .
:John :hasWife :Mary .
```

What is inferred?

Answer:

1. `:Mary :hasHusband :John` (inverseOf)
2. `:John rdf:type :Man` (domain of hasWife)
3. `:Mary rdf:type :Woman` (range of hasWife)

8 Quick Reference Cheat Sheet

CLASS TEST – Key Concepts	
Knowledge Graph	Network of nodes (things) and edges (relationships)
Triple	(Subject, Predicate, Object) – the atomic unit of RDF
IRI	Unique web identifier for a resource
Literal	A plain value (string, integer, date) with a datatype
Blank Node	A node without an IRI (anonymous)
RDF	Standard language for knowledge graphs using triples
Turtle	Compact RDF format: <code>subject predicate object .</code>
a	Shortcut for <code>rdf:type</code> in Turtle
; , , , .	Same-subject-new-pred ; same-pred-new-obj , end .
Bag / Seq / Alt	Unordered / Ordered / Pick-one containers (OPEN)
Collection	Closed group (<code>parseType="Collection"</code>)
RDFS	Schema layer: classes, hierarchies, domain, range
rdfs:subClassOf	“A is a kind of B” (transitive)
rdfs:domain	Subject must be this class
rdfs:range	Object must be this class/datatype
Inference	Discovering implicit facts from rules
SKOS Concept	A “unit of thought” in a vocabulary
skos:broader	More general concept (hierarchical)
skos:narrower	More specific concept (hierarchical)
skos:related	Associated concept (non-hierarchical, symmetric)
broaderTransitive	Explicitly transitive version of broader
Disjoint (SKOS)	related is disjoint with broader/narrower

FINAL EXAM ONLY – OWL Additions	
OWL	Full ontology language with strong semantics
TBox / ABox	Ontology (schema) / Data (instances)
owl:equivalentClass	Two classes are the same
owl:disjointWith	Two classes cannot overlap
owl:inverseOf	Property A is reverse of property B
Symmetric	$A \rightarrow B$ means $B \rightarrow A$
Transitive (OWL)	$A \rightarrow B$, $B \rightarrow C$ means $A \rightarrow C$
Functional	At most one value per subject
Protégé	Free ontology editor for OWL
Reasoner	Algorithm that checks consistency and finds inferences

End of Week 2 Study Notes

F20BD/F21BD – Big Data Management | Heriot-Watt University
Good luck with your class test and final exam!